

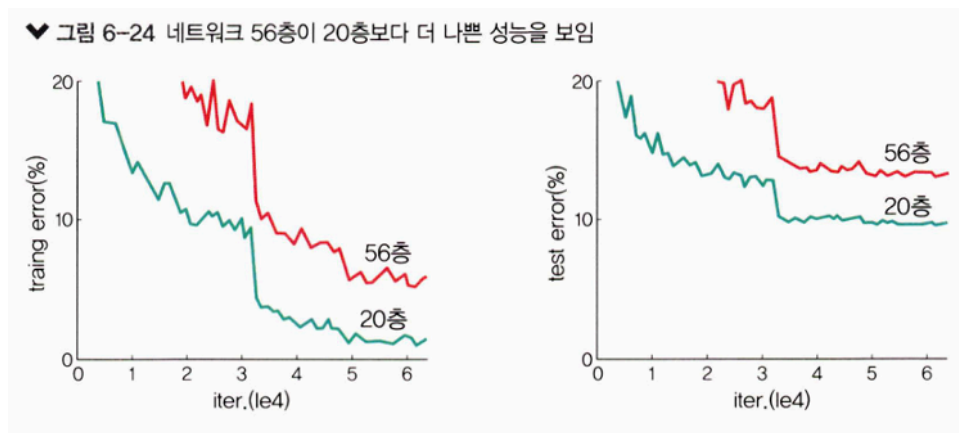
6주차 예습과제

📖 딥러닝 파이토치 교과서 6장 6.1.5-

p312-357

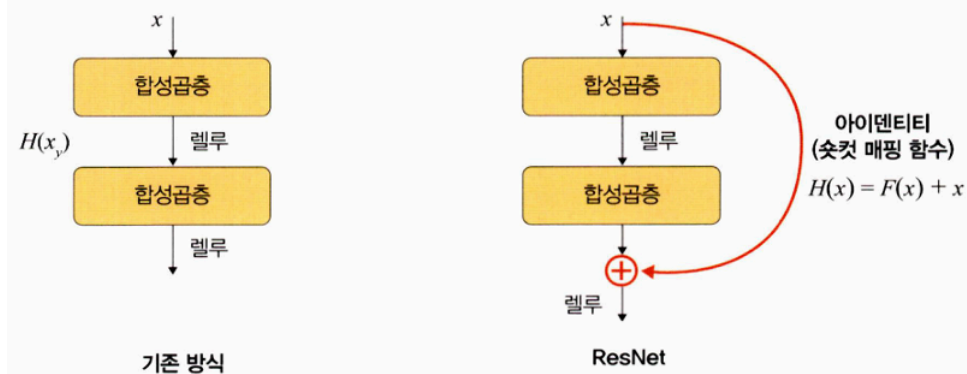
6.1.5 ResNet

- 마이크로소프트에서 개발한 알고리즘으로 "Deep Residual Learning for Image Recognition"이라는 논문에서 발표
- 깊어진 신경망을 효과적으로 학습하기 위한 방법으로 레지듀얼(residual) 개념을 고안한 거싱 핵심
 - 논문에 따르면 신경망은 깊이가 깊어질수록 성능이 좋아지다가 일정한 단계에 다다르면 오히려 성능이 나빠진다고 함



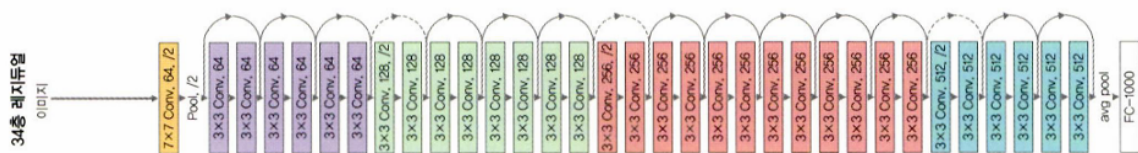
- 네트워크 깊이가 깊다고 해서 무조건 성능이 좋아지지 않는
- 이 문제를 해결하기 위해 레지듀얼 블록(residual block)을 도입
- 레지듀얼 블록은 기울기가 잘 전파될 수 있도록 일종의 숏컷(shortcut, skip connection)을 만들어 줌
- 이러한 개념이 필요한 이유: GoogLeNet은 층이 총 22개로 구성된 것에 비해 ResNet은 층이 총 152개로 구성되어 기울기 소멸 문제가 발생할 수 있기 때문
 - 숏컷을 두어 기울기 소멸 문제를 방지

♥ 그림 6-25 ResNet 구조



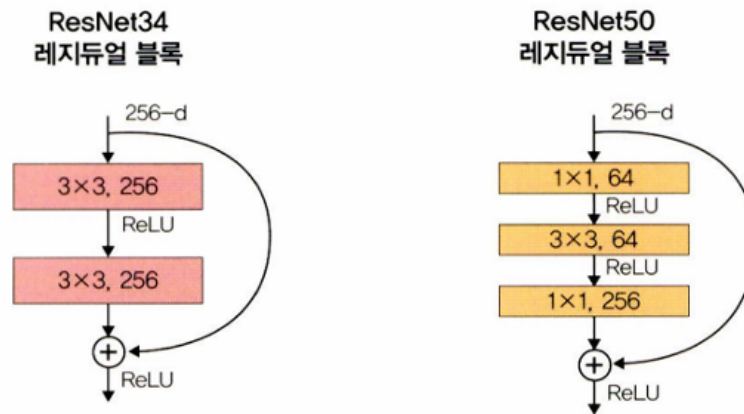
- 블록(block)
 - 계층의 묶음
 - 합성곱층을 하나의 블록으로 묶은 것

♥ 그림 6-26 ResNet 모델 전체 네트워크



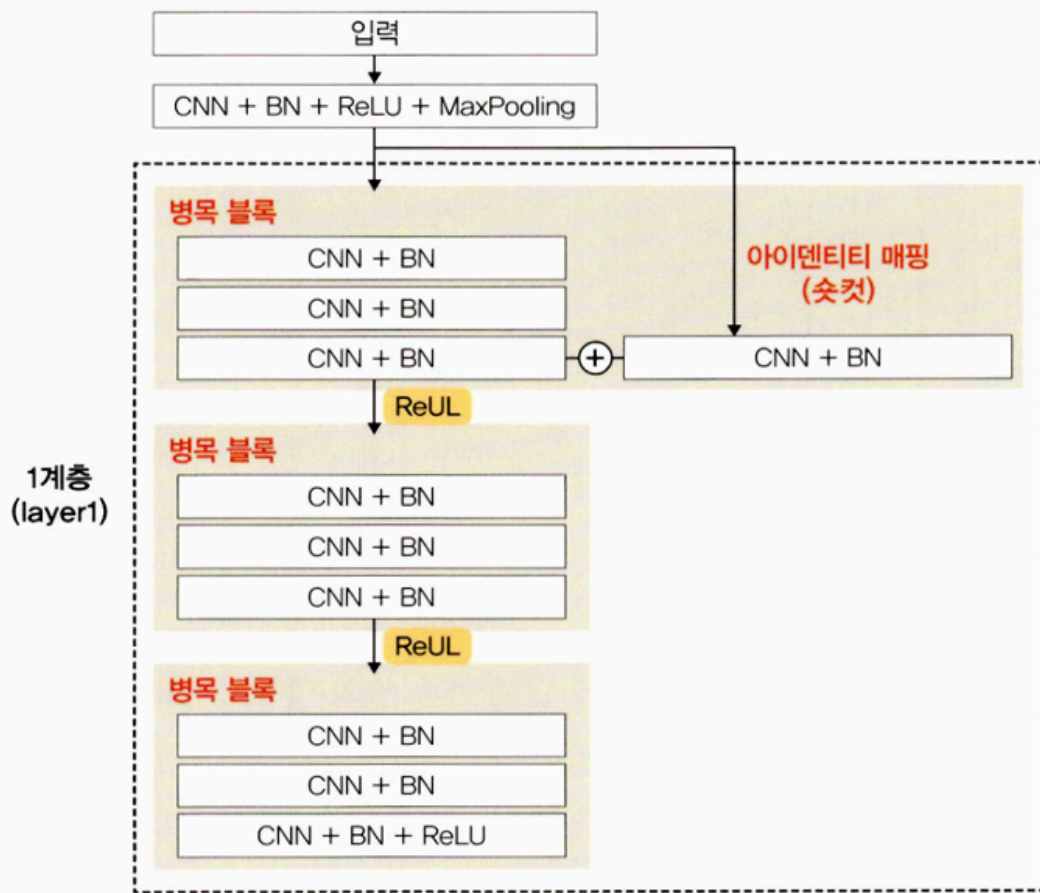
- 묶인 계층들을 하나의 레지듀얼 블록이라고 함
 - 레지듀얼 블록을 여러 개 쌓은 것을 ResNet이라고 함
 - 하지만 이렇게 계층을 계속해서 쌓아 늘리면 파라미터 수가 문제가 됨
 - 계층이 깊어질수록 파라미터는 증가
 - Ex) ResNet34는 합성곱층이 34개와 16개의 블록으로 구성되어 있음
 - 첫 번째 플록의 파라미터가 1152K라면 전체 파라미터 수는 2만 1282K
 - 이와 같이 계층의 깊이가 깊어질수록 파라미터는 무제한으로 커짐
- 이 문제 해결하기 위해 병목 블록(bottleneck block)을 둬

▼ 그림 6-27 기본 블록과 병목 블록



- 그림에서 ResNet34는 기본블록, ResNet50은 병목 블록을 사용
- 기본 블록은 파라미터 수 39.3216M, 병목 블록은 파라미터 수 6.9632M
- 깊이가 깊어졌음에도 파라미터 수는 감소
- 병목 블록을 사용하면 파라미터 수가 감소하는 효과 줄 수 있음
 - ResNet34와 달리 ResNet50에서는 3×3 합성곱층 앞뒤로 1×1 합성곱층이 붙어있는데, 1×1 합성곱층의 채널 수를 조절하면서 차원을 줄였다 늘리는 것이 가능하기 때문에 파라미터 수를 줄일 수 있음
 - 이 부분이 병목과 같다고 하여 병목 블록이라고 함
- 아이덴티티 매핑(identity mapping)(혹은 숏컷, 스킵 연결)
 - 위 그림 아래쪽의 + 기호 부분을 아이덴티티 매핑이라고 함(기본, 병목 블록 모두에서 사용됨)
 - 아이덴티티 매핑은 입력 x 가 어떤 함수를 통과하더라도 다시 x 라는 형태로 출력되도록 함

▼ 그림 6-28 아이덴티티 매핑(숏컷)



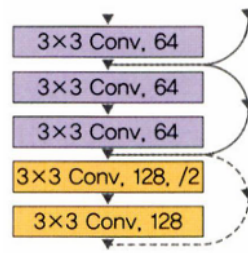
```
def forward(self, x):
    i = x
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.conv2(x)
    x = self.bn2(x)

    if self.downsample is not None:
        i = self.downsample(i) #다운샘플 적용

    x += i #아이덴티티 매핑 적용
    x = self.relu(x)
    return x
```

- 다운샘플(downsample)
 - 특성 맵 크기를 줄이기 위한 것으로 풀링과 같은 역할을 함

▼ 그림 6-29 ResNet 네트워크의 일부

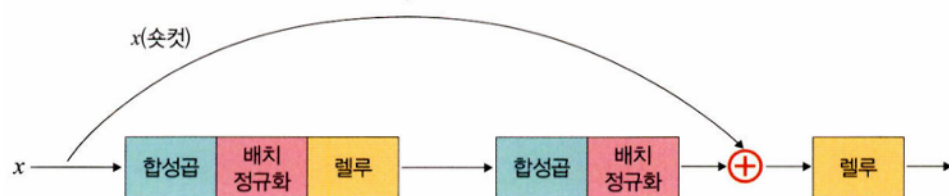


- 보라색 영역의 첫 번째 블록에서 특성 맵의 형상이 (28, 28, 64)였다면 세 번째 블록의 마지막 합성곱층을 통과하고 아이덴티티 매핑까지 완료된 특성 맵의 형상도 (28, 28, 64)
- 노란색 영역의 시작 지점에서는 채널 수가 128로 늘어났고, /2라는 것으로 보아 첫 번째 블록에서 합성곱층의 스트라이드가 2로 들어가 (14, 14, 128)로 바뀐다는 것 알 수 있음

→ 즉, 보라색과 노란색의 형태가 다른데 이들 간의 형태를 맞추지 않으면 아이덴티티 매핑을 할 수 없게 되므로 아이덴티티에 대해 다운샘플이 필요함

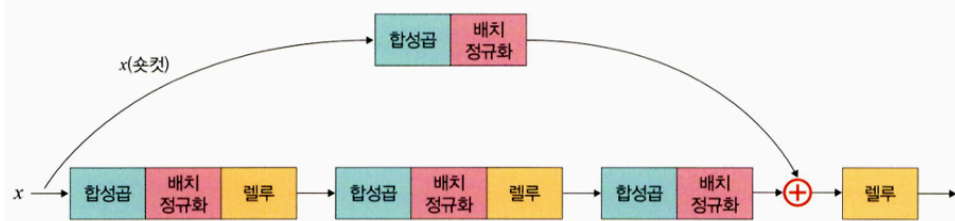
- 입력과 출력의 형태를 같도록 맞추어 주기 위해서는 스트라이드 2를 가진 1x1 합성곱 계층을 하나 연결해주면 됨
- 이와 같이 입력과 출력의 차원이 같은 것을 아이덴티티 블록이라고 함

▼ 그림 6-31 아이덴티티 블록



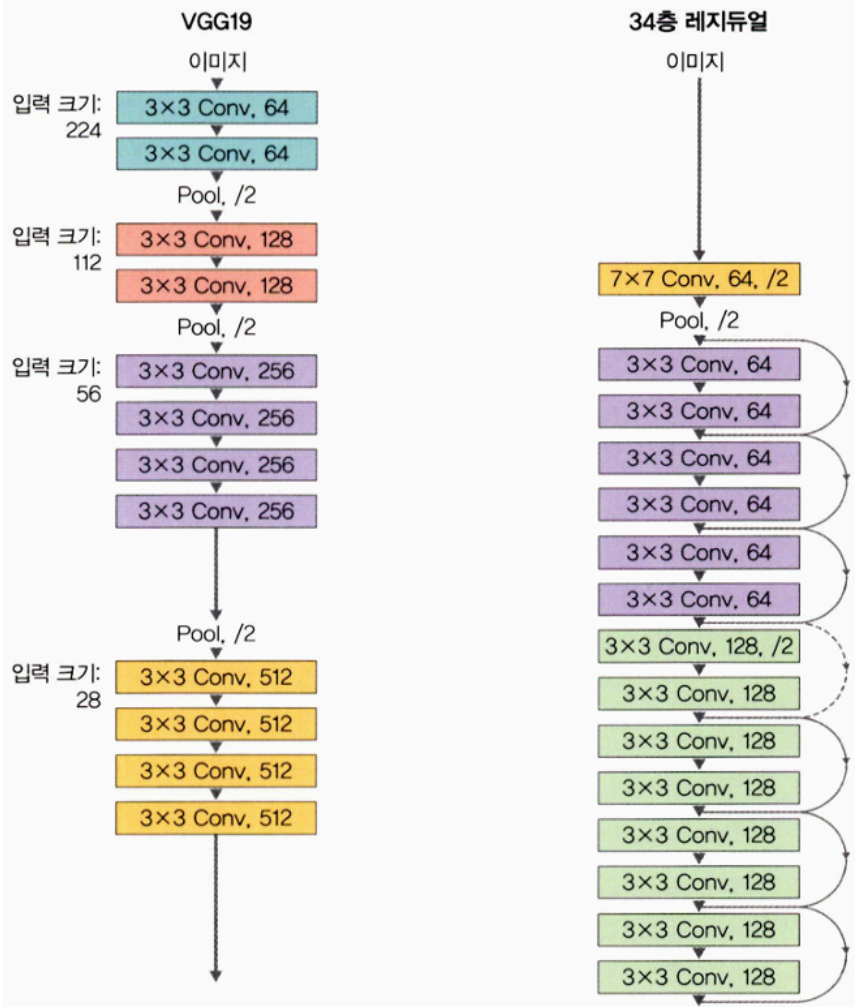
- 입력 및 출력 차원이 동일하지 않고 입력의 차원을 출력에 맞추어 변경해야 하는 것을 프로젝션 숏컷 혹은 합성곱 블록이라고 함

▼ 그림 6-30 합성곱 블록



- 정리하자면 ResNet은 기본적으로 VGG19 구조를 뼈대로 하며, 거기에 합성곱층들을 추가해서 깊게 만든 후 숏컷들을 추가하는 것

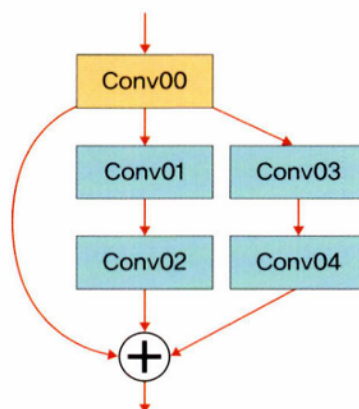
▼ 그림 6-32 VGG19와 ResNet 비교



▼ 레지듀얼 분기 (코드 6-75)

- 레지듀얼 분기란 프로그램에서의 조건에 따라 A, B, C 등으로 분기하는 것과 같음

▼ 그림 6-33 레지듀얼 분기



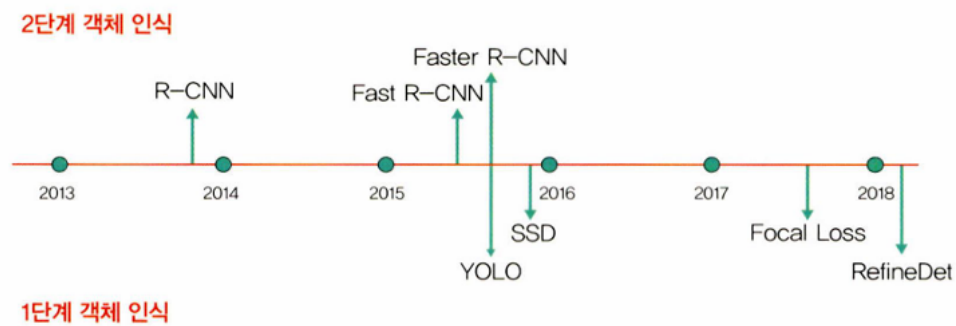
6.2 객체 인식을 위한 신경망

- 객체 인식(object detection)은 이미지나 영상 내에 있는 객체를 식별하는 컴퓨터 비전 기술
- 이미지나 영상 내에 있는 여러 개체에 대해 각 객체가 무엇인지 분류하는 문제와 그 객체 위치가 어디인지 박스(bounding box)로 나타내는 위치 검출(localization) 문제를 다루는 분야

객체 인식 = 여러 가지 객체에 대한 분류 + 객체의 위치 정보를 파악하는 위치 검출

- 크게 1단계 객체 인식(1-stage detector)과 2단계 객체 인식(2-stage detector)으로 나눌 수 있음

▼ 그림 6-35 1단계 객체 인식 vs 2단계 객체 인식 흐름도



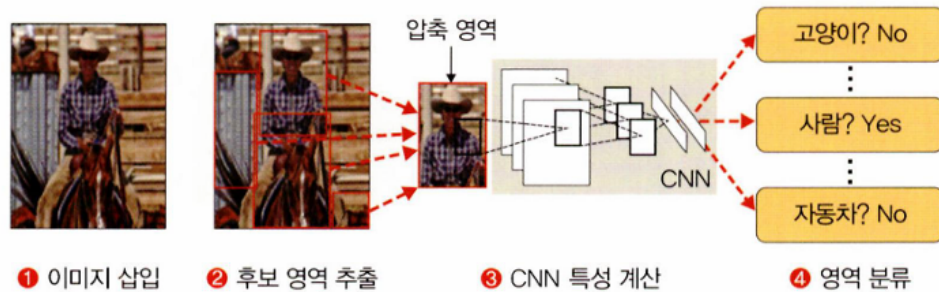
- 1단계 객체 인식은 분류와 위치 검출을 동시에 행하는 방법
 - 비교적 빠르지만 정확도가 낮음
 - YOLO(You Only Look Once) 계열과 SSD 계열 등
- 2단계 객체 인식은 분류와 위치 검출을 순차적으로 행하는 방법
 - 비교적 느리지만 정확도가 높음
 - R-CNN 계열 등

6.2.1 R-CNN

- 예전의 객체 인식 알고리즘들은 슬라이딩 윈도우 방식, 즉 일정한 크기를 가지는 윈도우를 가지고 이미지의 모든 영역을 탐색하면서 객체를 검출해내는 방식
- 알고리즘의 비효율성 때문에 많이 사용하지 않았으며, 현재는 선택적 탐색 알고리즘을 적용한 후보 영역(region proposal)을 많이 사용
- R-CNN(Region-based CNN)

- 이미지 분류를 수행하는 CNN과 이미지에서 객체가 있을 만한 영역을 제안해주는 후보 영역 알고리즘을 결합한 알고리즘

▼ 그림 6-36 R-CNN 학습 절차



1. 이미지를 입력으로 받음
2. 2000개의 바운딩 박스를 선택적 탐색 알고리즘으로 추출한 후 잘라 내고 (cropping), CNN 모델에 넣기 위해 같은 크기(227x227 픽셀)로 통일(wrapping)
3. 크기가 동일한 이미지 2000개에 각각 CNN 모델을 적용
4. 각각 분류를 진행하여 결과 도출

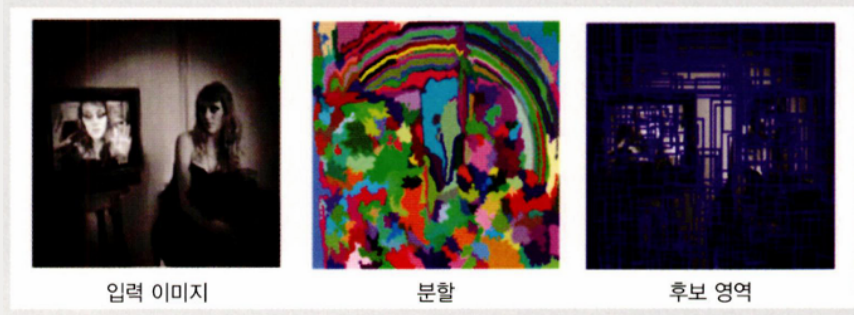
▼ 선택적 탐색

- 선택적 탐색은 객체 인식이나 검출을 위한 가능한 후보 영역(객체가 있을 만한 위치, 영역)을 알아내는 방법
- 분할 방식을 이용하여 시드(seed)를 선정하고 그 시드에 대한 완전 탐색을 적용

1단계. 초기 영역 생성(sub-segmentation)

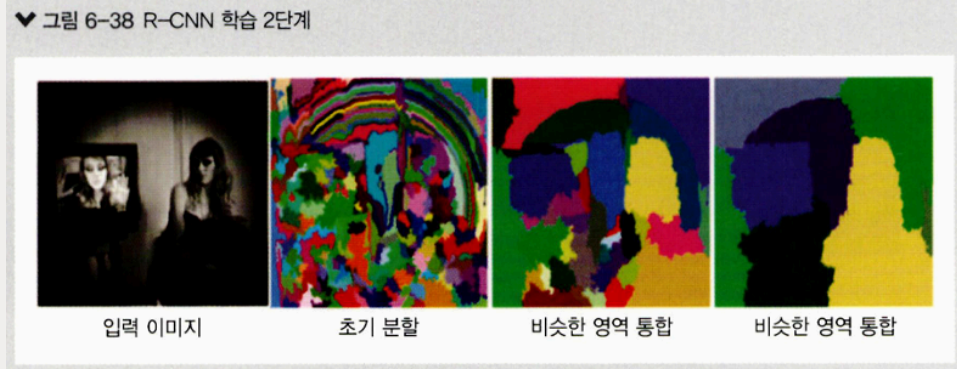
- 각각의 객체가 영역 한 개에 할당될 수 있도록 많은 초기 영역을 생성
- 입력된 이미지를 영역 다수 개로 분할하는 과정

▼ 그림 6-37 R-CNN 학습 1단계



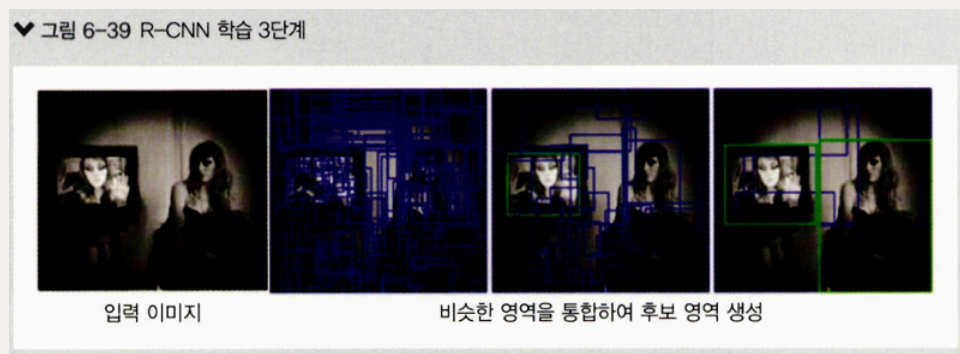
2단계. 작은 영역의 통합

- 1단계에서 영역 여러 개로 나눈 것들을 비슷한 영역으로 통합
- 탐욕(greedy) 알고리즘을 사용하여 비슷한 영역이 하나로 통합될 때까지 반복



3단계. 후보 영역 생성

- 2단계에서 통합된 이미지들을 기반으로 후보 영역(바운딩 박스)을 추출

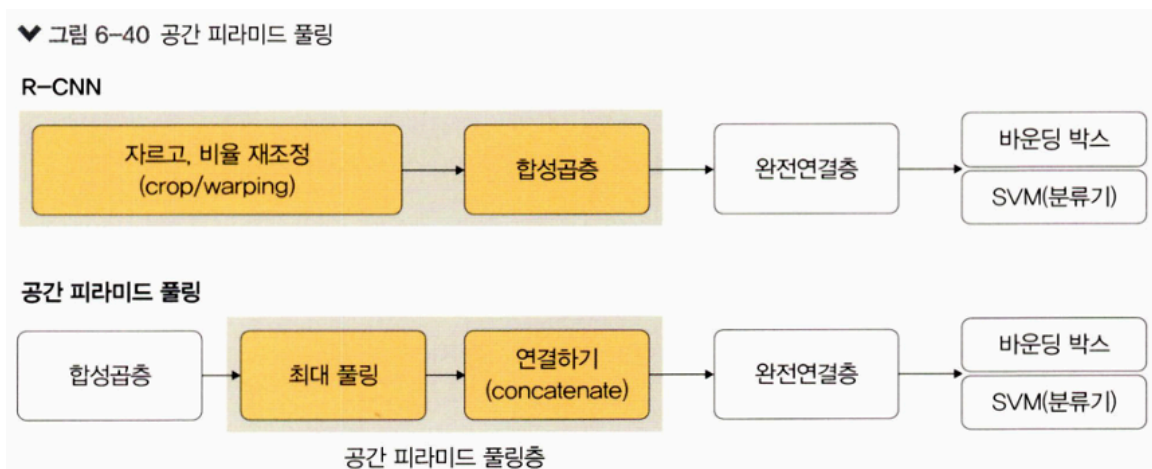


- **완전 탐색(exhaustive search):** 후보가 될 만한 대상의 크기 및 비율이 모두 다른 상황을 고려하여 후보 영역을 찾는 기법
 - **분할(segmentation):** 영상 데이터의 특성(색상, 모양, 무늬)등에 따라 분할하여 후보 영역을 선정하는 기법
 - **후보 영역(바운딩 박스):** 3D 객체의 형태를 모두 포함할 수 있는 최소 크기의 박스
 - **시드(seed):** 영상에서는 특정 기준점의 픽셀에서 점점 의미가 같은 영상 범위까지 픽셀을 확장해 나가면서 분할하는데, 이때 특정 기준점이 되는 픽셀
- R-CNN의 단점
 1. 앞서 언급한 세 단계의 복잡한 학습 과정
 2. 긴 학습 시간과 대용량 저장 공간
 3. 객체 검출(object detection) 속도 문제

→ 이러한 문제 해결하기 위해 Fast R-CNN이 생김

6.2.2 공간 피라미드 풀링

- 기존 CNN 구조들은 모두 완전연결층을 위해 입력 이미지를 고정해야 했고, 따라서 신경망을 통과시키려면 이미지를 고정된 크기로 자르거나(crop) 비율을 조정(wrap)해야 했음
- 하지만 이렇게 하면 물체의 일부분이 잘리거나 본래의 생김새와 달라지는 문제점
→ 이러한 문제 해결하고자 공간 피라미드 풀링(spatial pyramid pooling)을 도입

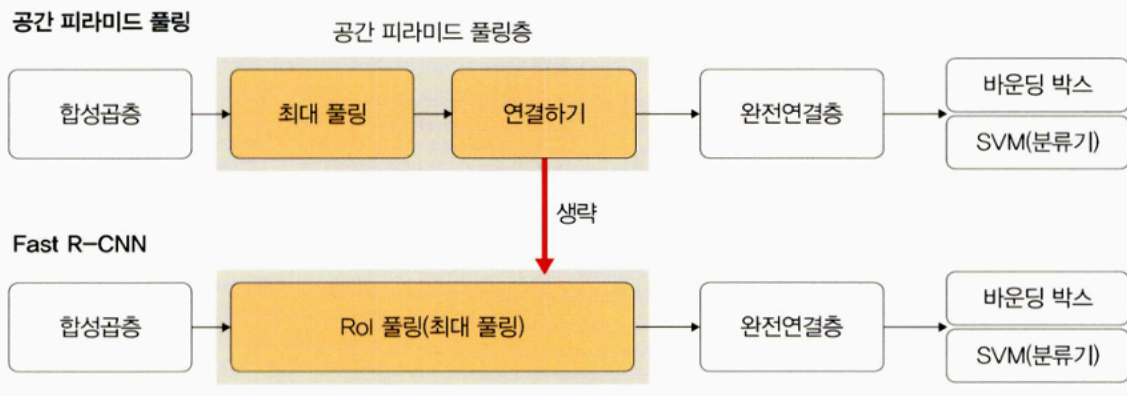


- 공간 피라미드 풀링은 입력 이미지의 크기에 관계없이 합성곱층을 통과시키고, 완전연결층에 전달되기 전에 특성 맵들을 동일한 크기로 조절해주는 풀링층을 적용하는 기법
 - 입력 이미지의 크기를 조절하지 않고 합성곱층을 통과시키기 때문에 원본 이미지의 특징이 훼손되지 않는 특성 맵을 얻을 수 있음
 - 이미지 분류나 객체 인식 같은 여러 작업에 적용할 수 있다는 장점

6.2.3 Fast R-CNN

- R-CNN의 문제: 바운딩 박스마다 CNN을 돌리고, 분류를 위한 긴 학습 시간 소요
- Fast R-CNN(Fast Region-based CNN)은 R-CNN의 속도 문제를 개선하기 위해 RoI 풀링을 도입
 - 선택적 탐색에서 찾은 바운딩 박스 정보가 CNN을 통과하면서 유지되도록 하고 최종 CNN특성 맵은 풀링을 적용하여 완전연결층을 통과하도록 크기를 조정
 - 바운딩 박스마다 CNN을 돌리는 시간을 단축할 수 있음

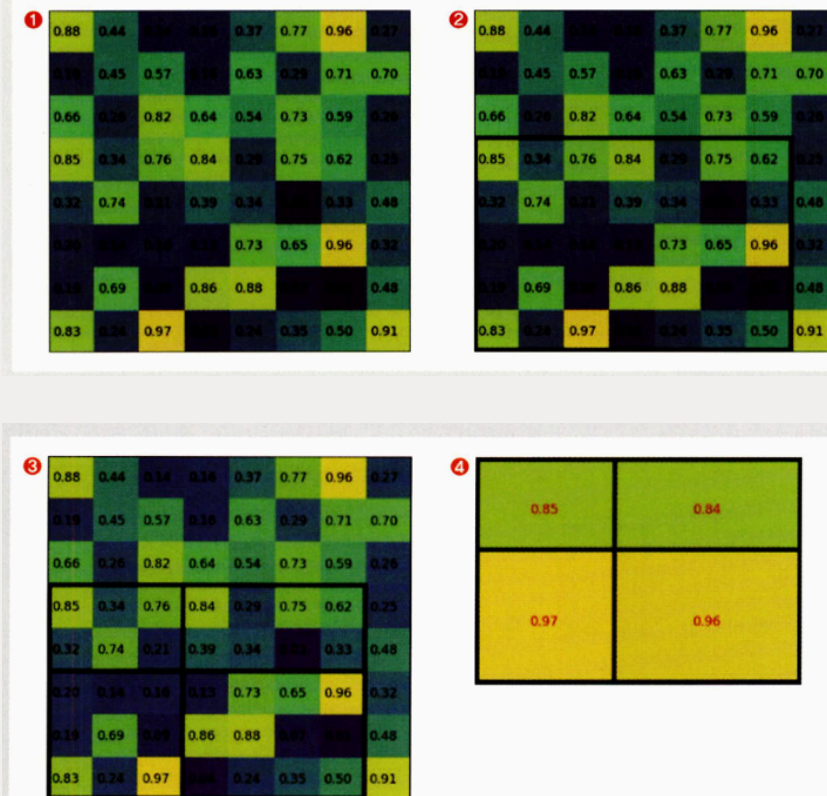
▼ 그림 6-41 Fast R-CNN



▼ RoI 풀링

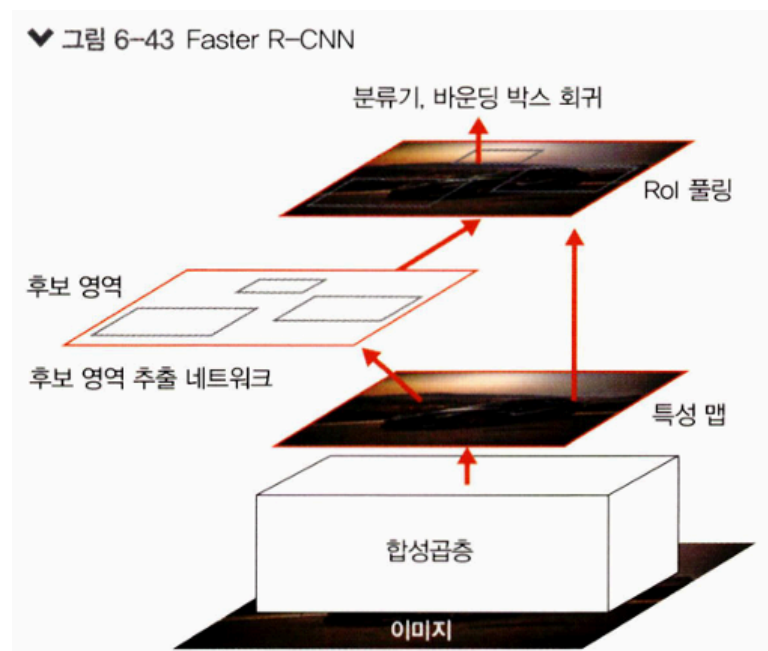
- RoI 풀링(RoI pooling)은 크기가 다른 특성 맵의 영역마다 스트라이드를 다르게 최대 풀링을 적용하여 결괏값 크기를 통일하게 맞추는 방법
- Ex) 박스 한 개가 픽셀 한 개를 뜻하는 특성 맵
 - 8×8 특성 맵(1)에서 선택적 탐색으로 뽑아냈던 7×5 후보 영역(2)이 있으며, 이것을 2×2로 만들기 위해 스트라이드($7/2=3$, $5/2=2$)로 풀링 영역(3)을 정하고 최대 풀링을 적용하면 2×2 결괏값(4)을 얻을 수 있음

▼ 그림 6-42 RoI 풀링

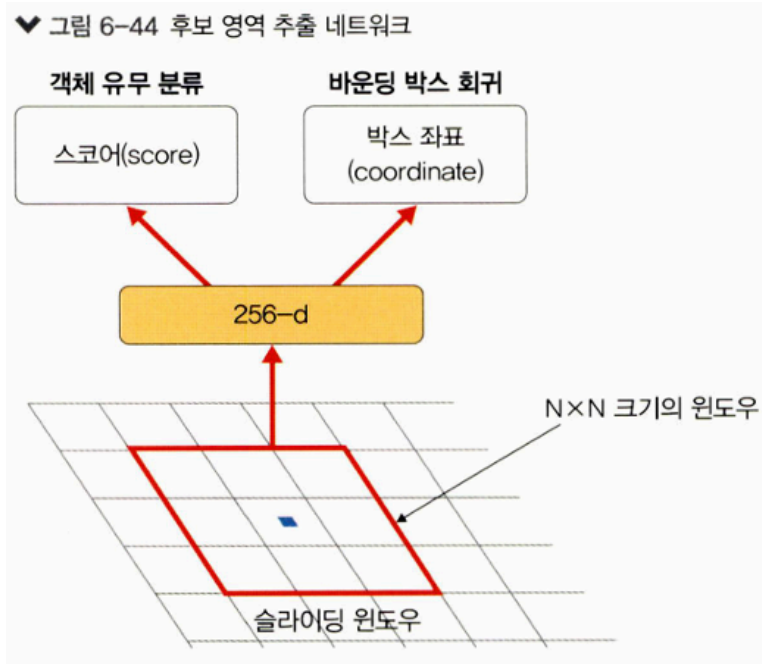


6.2.4 Faster R-CNN

- '더욱 빠른' 객체 인식을 수행하기 위한 네트워크
- 기존 Fast R-CNN 속도를 느리게 했던 후보 영역 생성을 CNN 내부 네트워크에서 진행할 수 있도록 설계
 - 기존 Fast R-CNN에 후보 영역 추출 네트워크(Region Proposal Network, RPN)를 추가한 것이 핵심
 - 외부의 느린 선택적 탐색(CPU로 계산) 대신 내부의 빠른 RPN(GPU로 계산)을 사용
- RPN은 마지막 합성곱층 다음에 위치하고, 그 뒤에 Fast R-CNN과 같이 RoI 풀링과 분류기(classifier), 바운딩 박스 회귀(bounding-box regression)가 위치함



- 후보 영역 추출 네트워크는 특성 맵 $N \times N$ 크기의 작은 윈도우 영역을 입력으로 받고, 해당 영역에 객체의 존재 유무 판단을 위해 이진 분류(binary classification)을 수행하는 작은 네트워크를 생성
 - R-CNN, Fast R-CNN에서 사용되었던 바운딩 박스 회귀(바운딩 박스가 이미지의 객체를 정확히 포착하여 포함시킬 수 있도록 조정해 주는 역할) 또한 위치 보정(좌표점 추론)을 위해 추가
 - 하나의 특성 맵에서 모든 영역에 대한 객체의 존재 유무를 확인하기 위해서 슬라이딩 윈도우 방식으로 앞서 설계한 작은 윈도우 영역($N \times N$ 크기)을 이용하여 객체를 탐색

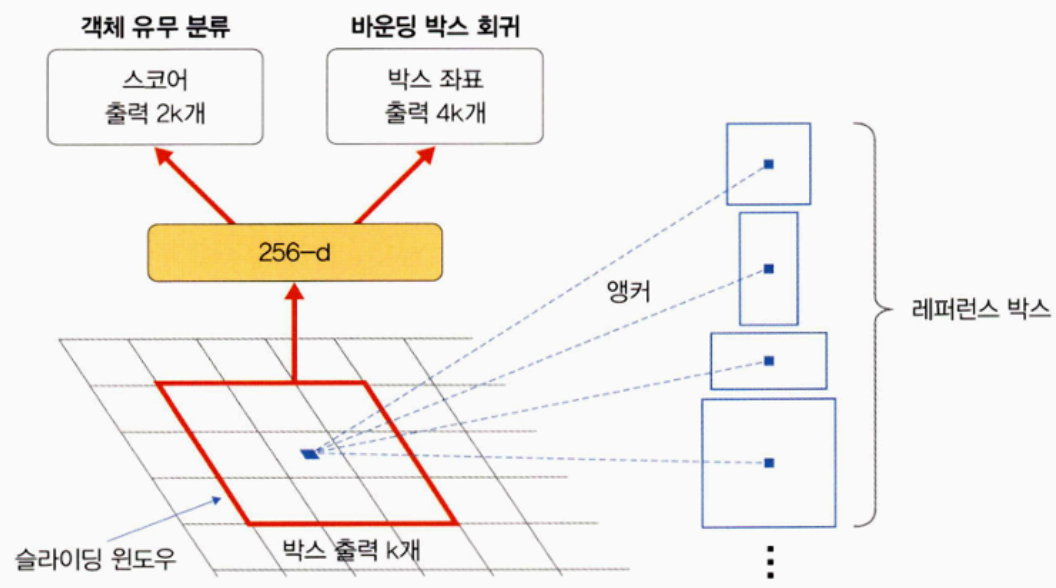


- 후보 영역 추출 네트워크는 이미지에 존재하는 객체들의 크기와 비율이 다양하기 때문에 고정된 $N \times N$ 크기의 입력만으로는 다양한 크기와 비율의 이미지를 수용하기 어려운 단점 있음

→ 앵커(anchor): 이러한 단점 보완하기 위해 여러 크기와 비율의 레퍼런스 박스 k 개를 미리 정의, 각각의 슬라이딩 윈도우 위치마다 박스 k 개를 출력하도록 설계

- 즉, 후보 영역 추출 네트워크의 출력 값은 모든 앵커 위치에 대해 각각 객체와 배경을 판단하는 $2k$ 개의 분류에 대한 출력과 x, y, w, h 위치 보정 값을 위한 $4k$ 개의 회귀 출력을 가짐
- Ex) 특성 맵 크기가 $w \times h$ 라면 하나의 특성 맵에 앵커가 총 $w \times h \times k$ 개 존재

▼ 그림 6-45 앵커



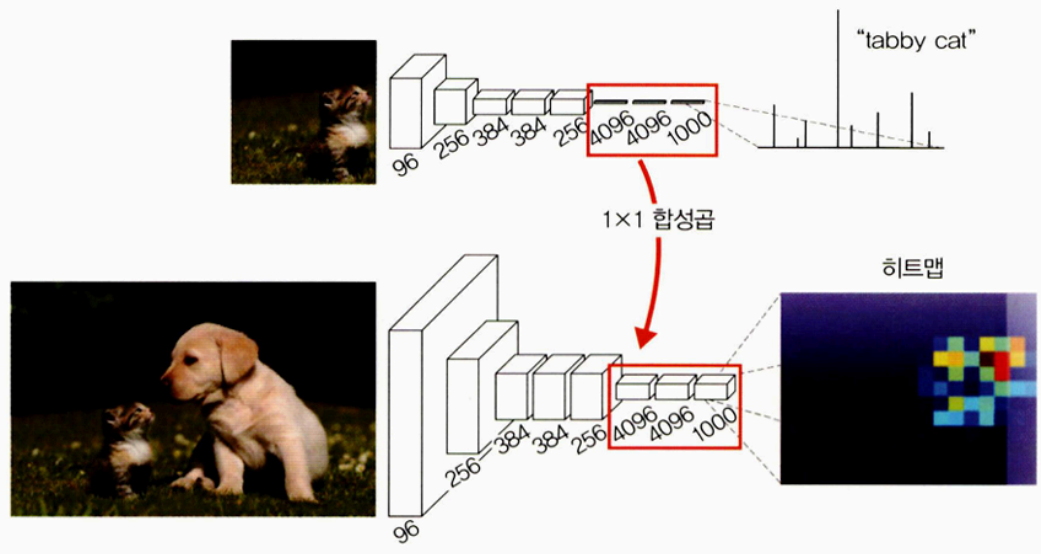
6.3 이미지 분할을 위한 신경망

- 이미지 분할(image segmentation)은 신경망을 훈련시켜 이미지를 픽셀 단위로 분할하는 것
- 이미지를 픽셀 단위로 분할하여 이미지에 초함된 객체를 추출
- 대표적 네트워크로 완전 합성곱 네트워크, 합성곱 & 역합성곱 네트워크, U0Net, PSPNet, DeepLabv3, DeepLabv3+가 있음

6.3.1 완전 합성곱 네트워크

- 완전연결층의 한계: 고정된 크기의 입력만 받아들임, 완전연결층을 거친 후에는 위치 정보가 사라짐
→ 이러한 문제를 해결하기 위해 완전연결층을 1x1 합성곱으로 대체하는 것이 완전 합성곱 네트워크
- 완전 합성곱 네트워크(Fully Convolutional Network, FCN)는 이미지 분류에서 우수한 성능을 보인 CNN 기반 모델(AlexNet, VGG16, GoogLeNet)을 변형시켜 이미지 분할에 적합하도록 만든 네트워크
 - Ex) AlexNet 아래쪽에서 사용되었던 완전연결층 세 개를 1x1 합성곱으로 변환하면 위치 정보가 남아 있기 때문에 히트맵 그림과 같이 고양이의 위치를 확인할 수 있음

♥ 그림 6-46 완전 합성곱 네트워크

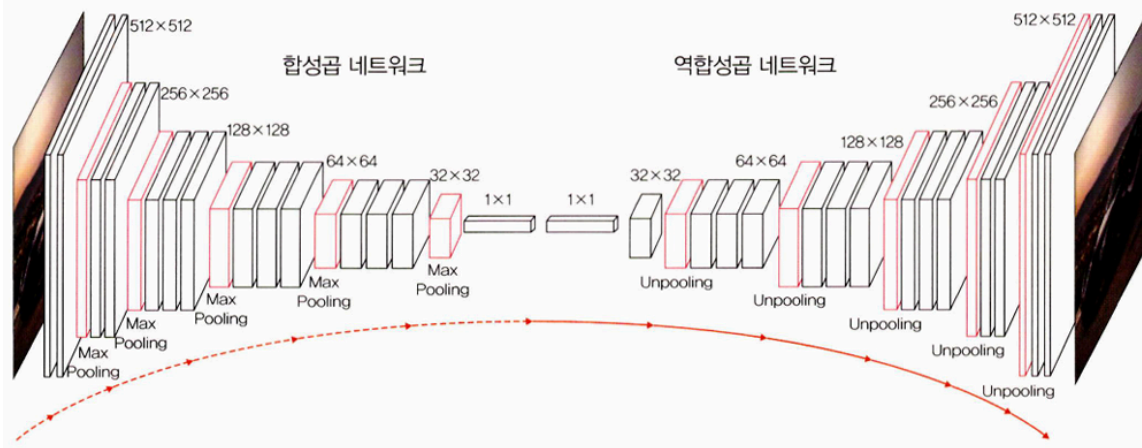


- 위치 정보가 보존된다는 장점, 합성곱층으로 사용되기 때문에 입력 이미지에 대한 크기 제약이 사라지는 장점

6.3.2 합성곱 & 역합성곱 네트워크

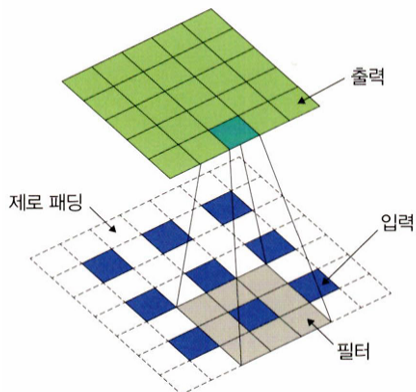
- 완전 합성곱 네트워크의 단점
 - 여러 단계의 합성곱층과 풀링층을 거치면서 해상도가 낮아짐
 - 낮아진 해상도를 복원하기 위해 업 샘플링(최종 이미지의 크기가 입력 이미지의 크기와 같도록 하는 것) 방식을 사용하기 때문에 이미지의 세부 정보들을 잃어버리는 문제가 발생
- 이를 해결하기 위해 역합성곱 네트워크를 도입한 것이 합성곱 & 역합성곱 네트워크 (convolutional & deconvolutional network)
- 역합성곱은 CNN의 최종 출력 결과를 원래의 입력 이미지와 같은 크기로 만들고 싶을 때 사용
 - 시맨틱 분할(semantic segmentation, 이미지 내에 있는 물체들을 의미 있는 단위로 분할하는 것) 등에 활용할 수 있음
 - 역합성곱을 업 샘플링(upsampling)이라고도 함

▼ 그림 6-47 합성곱 & 역합성곱 네트워크



- 역합성곱은 특성 맵 크기를 증가시키는 방식으로 동작(CNN에서 합성곱층이 합성곱을 사용하여 특성 맵 크기를 줄이는 것과 반대)
 1. 각각의 픽셀 주위에 제로 패딩을 추가
 2. 이렇게 패딩된 것에 합성곱 연산을 수행

▼ 그림 6-48 역합성곱 진행 방식



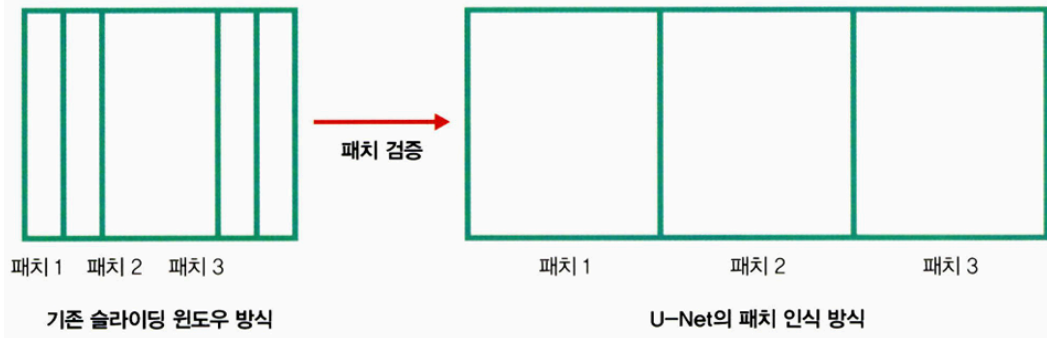
→ 아래쪽의 파란색 픽셀이 입력, 초록색 픽셀이 출력

- 파란색 픽셀 주위로 흰색 제로 패딩을 수행
- 회색 필터로 합성곱 연산을 수행하면 초록색이 출력됨

6.3.3 U-Net

- 바이오 메디컬 이미지 분할을 위한 합성곱 신경망
- U-Net의 특징
 - 속도가 빠르다
 - 기존 슬라이딩 윈도우 방식은 이전 패치(patch, 이미지 인식 단위)에서 검증이 끝난 부분을 다음 패치에서 또 검증하기 때문에 속도가 느림
 - U-Net은 이미 검증이 끝난 패치는 건너뛰기 때문에 속도가 빠름

▼ 그림 6-49 슬라이딩 윈도우 방식



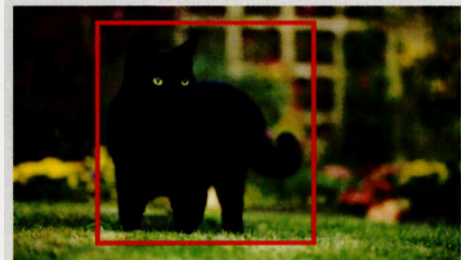
○ **트레이드오프(trade-off)에 빠지지 않는다**

- 일반적으로 패치 크기가 커진다면 넓은 범위의 이미지를 인식하는 데 뛰어나기 때문에 컨텍스트(context) 인식에 탁월
- 하지만 너무 넓은 범위를 한 번에 인식하기 때문에 지역화에는 한계가 있음
- U-Net은 컨텍스트 인식과 지역화 간의 트레이드오프 문제를 개선함

▼ **지역화**

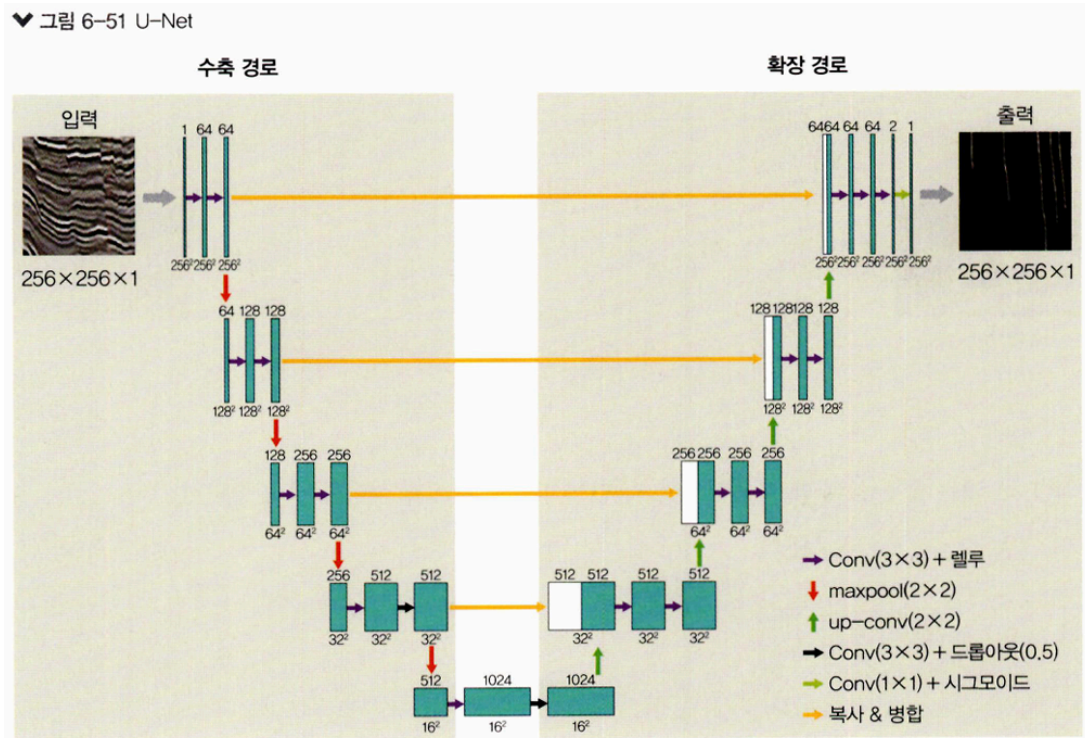
- 지역화(localization)는 이미지 안에 객체 위치 정보를 출력해주는 것
- 주로 바운딩 박스를 많이 사용
- 바운딩 박스의 네 꼭짓점 픽셀 좌표가 출력되는 것이 아닌 왼쪽 위(left top), 오른쪽 아래(right bottom) 좌표를 출력

▼ 그림 6-50 지역화



- U-Net 구조
 - FCN을 기반으로 구축
 - 수축 경로(contracting path)와 확장 경로(expansive path)로 구성
 - 수축 경로: 컨텍스트를 포착
 - 확장 경로: 특성 맵을 업 샘플링 하고 수축 경로에서 포착한 특성 맵의 컨텍스트와 결합하여 정확한 지역화를 수행
 - U-Net은 3x3 합성곱이 주를 이룸

- 각 합성곱 블록은 3x3 합성곱 두 개로 구성, 그 사이에 드롭아웃이 있음
- 각 블록은 최대 풀링을 이용하여 크기를 줄이면서 다음 블록으로 넘어감
- Ex) 아래 그림의 왼쪽 수축 경로에서의 블록은 3x3 합성곱 두 개로 구성된 것이 네 개가 있는 형태
- 아래 그림의 오른쪽 확장 경로에서는 합성곱 블록에 up-conv를 앞에 붙임
- 수축 과정에서 줄어든 크기를 다시 키워가면서 합성곱 블록을 이용하는 형태

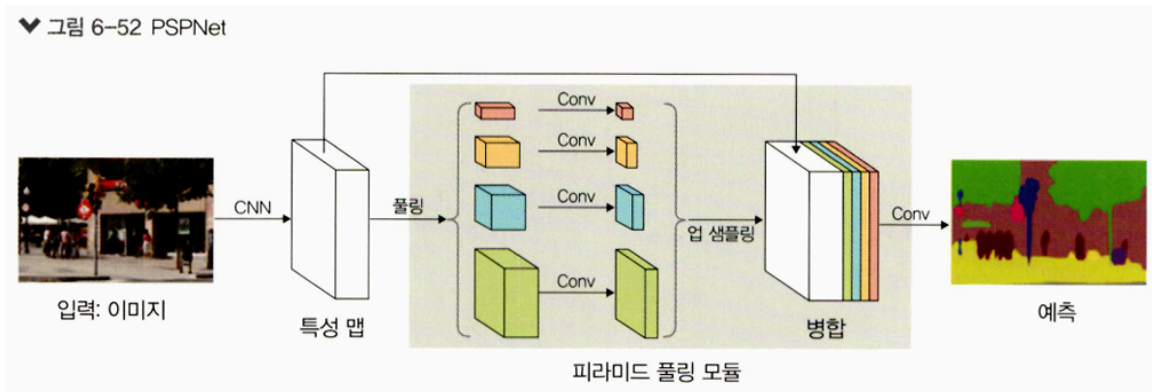


- 즉, 크기가 다양한 이미지의 객체를 분할하기 위해 크기가 다양한 특성 맵을 병합할 수 있도록 다운 샘플링과 업 샘플링을 순서대로 반복하는 구조

6.3.4 PSPNet

- PSPNet(Pyramid Scene Parsing Network)은 CVPR(The IEEE Conference on Computer Vision and Pattern Recognition) 2017에서 발표된 시멘틱 분할 알고리즘
- 완전연결층의 한계를 극복하기 위해 피라미드 풀링 모듈을 추가
- PSPNet 훈련 과정
 1. 이미지 출력이 서로 다른 크기가 되도록 여러 차례 풀링
 - 1×1, 2×2, 3×3, 6×6 크기로 풀링을 수행하는데, 이때 1×1 크기의 특성 맵은 가장 광범위한 정보를 담음

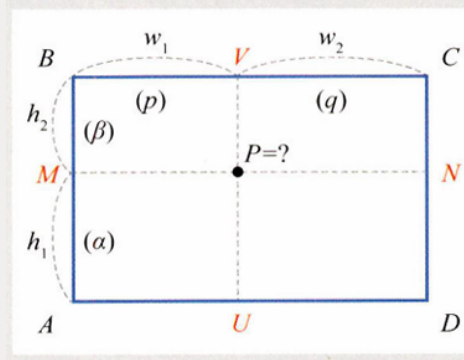
- 각각 다른 크기의 특성맵은 서로 다른 영역들의 정보를 담음
2. 이후 1x1 합성곱을 사용하여 채널 수를 조정
 - 풀링층 개수를 N이라고 할 때 (출력 채널 수) = (입력 채널 수)/N
 3. 이후 모듈의 입력 크기에 맞게 특성 맵을 업 샘플링
 - 이 과정에서 양선형 보간법(bilinear interpolation)이 사용됨
 4. 원래의 특성 맵과 1~3 과정에서 생성한 새로운 특성 맵들을 병합



▼ 양선형 보간법

- 보간법(interpolation)
 - 화소 값을 할당받지 못한 영상(Ex-영상의 빈 공간)의 품질은 안 좋을 수 밖에 없는데, 이때 빈 화소에 값을 할당하여 좋은 품질의 영상을 만드는 방법
 - 선형 보간법(linear interpolation)과 양선형 보간법(bilinear interpolation)이 있음
- 선형 보간법
 - 원시 영상의 화소 값 두 개를 사용하여 원하는 좌표에서 새로운 화소 값을 계산하는 방법
- 양선형 보간법
 - 화소당 선형 보간을 세 번 수행하여, 새롭게 생성된 화소는 가장 가까운 화소 네 개에 가중치를 곱한 값을 합해서 얻음

▼ 그림 6-53 양선형 보간법



Ex) 직사각형의 네 꼭짓점에 값이 주어져 있을 때 사각형 내부에 있는 임의의 점 P에 대한 값 추정

- 점 P에서 x축 방향으로 거리를 w_1, w_2 , y축 방향으로 거리를 h_1, h_2
- 알려진 네 점에서 데이터 값을 A, B, C, D 라고 할 때, 양선형 보간법에 따라 점 P의 값은 다음과 같이 계산됨(단 $\alpha = h_1/(h_1 + h_2)$, $\beta = h_2/(h_1 + h_2)$, $p = w_1/(w_1 + w_2)$, $q = w_2/(w_1 + w_2)$)

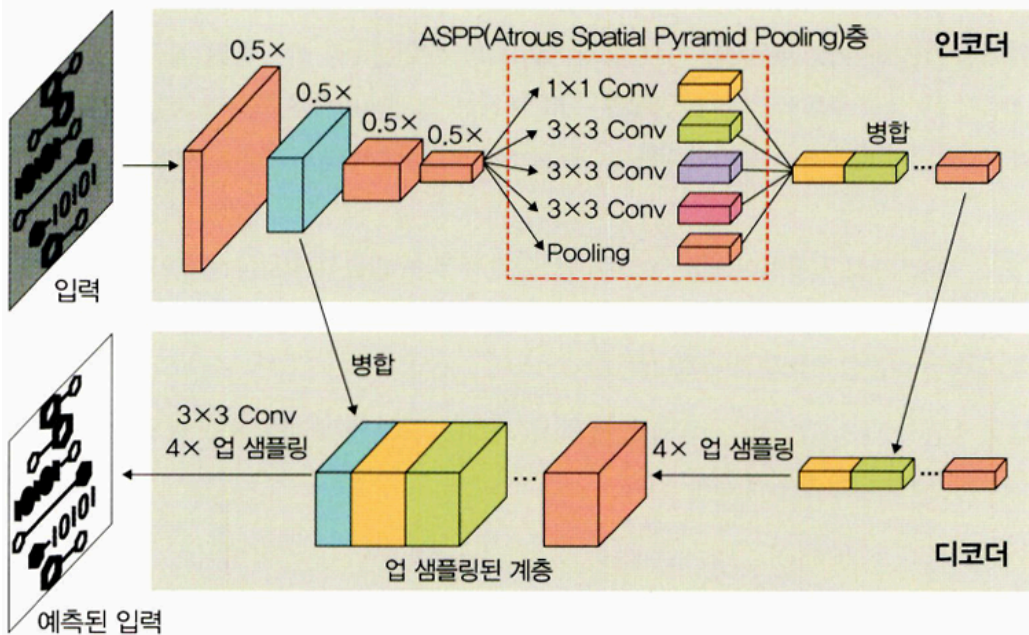
$$P = q(\beta A + \alpha B) + p(\beta D + \alpha C)$$

$$= q\beta A + q\alpha B + p\beta D + p\alpha C$$

6.3.5 DeepLabv3/DeepLabv3+

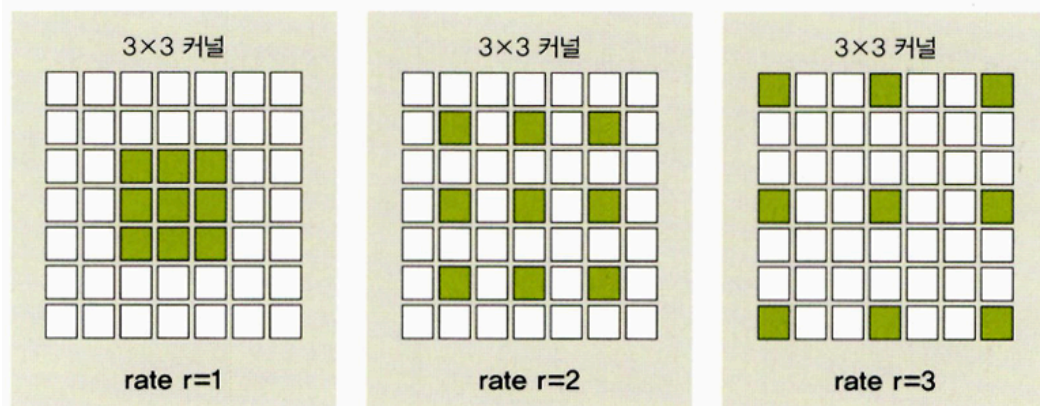
- 완전연결층의 단점을 보완하기 위해 Atrous 합성곱을 사용하는 네트워크
- 인코더와 디코더 구조를 가지며, 일반적으로 인코더-디코더 구조에서는 불가능했던 인코더에서 추출된 특성 맵의 해상도를 Atrous 합성곱을 도입하여 제어할 수 있도록 함

▼ 그림 6-54 DeepLab의 인코더-디코더 구조



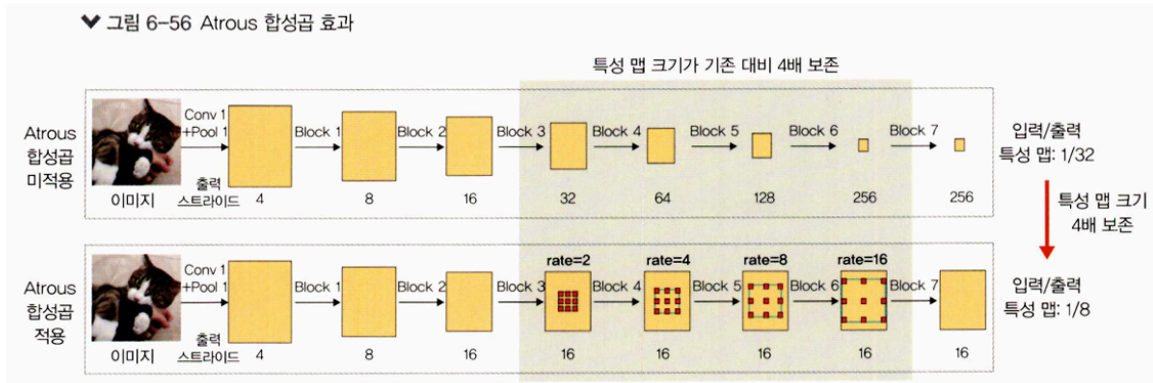
- Atrous 합성곱은 필터 내부에 빈 공간을 둔 채로 작동
 - 얼마나 많은 빈 공간을 가질지 결정하는 파라미터 **rate**
 - rate $r=1$ 일 경우 기존 합성곱과 동일하게 빈 공간을 가짐
 - r 이 커질수록 빈 공간은 더 많아짐

▼ 그림 6-55 Atrous 합성곱



- 이미지 분할에서 높은 성능을 내려면 수용 영역(receptive field)의 크기가 중요함
 - 수용 영역을 확대하여 특성을 찾는 범위를 넓게 해주기 때문
- Atrous 합성곱을 활용하면 파라미터 수를 늘리지 않으면서도 수용 영역을 크게 키울 수 있기 때문에 이미지 분할 분야에서 많이 사용

- 다음 그림과 같이 일반적인 CNN을 적용하면 출력은 입력에 비해 1/32로 줄어들지만, Atrous 합성곱을 적용하면 1/8로 줄어듦
→ 특성 맵 크기가 기존 대비 4배 보존된 것을 확인할 수 있음



▼ 수용 영역

- 외부 자극이 전체에 영향을 주는 것이 아니라 특정 영역에만 영향을 준다는 의미
- 영상에서 특정 위치에 있는 픽셀들은 그 주변에 있는 일부 픽셀과 연관성은 높지만 거리가 멀어질수록 그 영향은 감소하게 됨
- 영상 전체 영역에 대해 서로 동일한 중요도를 부여하여 처리하는 대신, 특정 범위를 한정해서 처리하여 효과적으로 훈련을 수행하는 것이 수용 영역